The Language Is the Prompt:

Why Elixir Nearly Doubled Python on AutoCodeBench, and What 3,920 Tasks Reveal About LLM-Friendly Language Design

Günther Brunner
CyberAgent, Inc.

contact@guntherbrunner.com
https://guntherbrunner.com

March 2026

The limits of my language mean the limits of my world.
— Ludwig Wittgenstein, Tractatus Logico-Philosophicus (1922)

Abstract

Elixir, a low-share language on the Erlang VM, scores 87.4% Pass@1 on AutoCodeBench while Python scores 43.9%. That is not merely a leaderboard curiosity. It is a clue about what code-generating models reward. This paper reproduces the 20-language AutoCodeBench result with GPT-5.4 (medium reasoning), shows that the advantage survives difficulty controls, and then asks the question the leaderboard itself cannot answer: *why*? Across four completed active-ablation suites (2,970 condition-level evaluations), a 144-row exact-task multilingual panel, and a 384-row fractional-factorial follow-up, the strongest within-Elixir causal signal is documentation structure, while the strongest portable cross-language intervention is explicit contract wording. Within Elixir, collapsing structured documentation cuts performance by roughly 40 percentage points. Across matched tasks in Elixir, Python, and TypeScript, the only portable intervention surviving Holm correction is explicit contract wording ($\Delta = +0.359$ on the 0–1 pass scale), with the largest gains in Python (+0.469) and TypeScript (+0.453).

We formalize these results as the *Explicitness Hypothesis*: languages and ecosystems that make intent, contracts, control flow, and data transformations locally visible reduce the *predictive burden* on a code-generating model. Elixir is the strongest case study in the current data, but the paper's durable claim is broader and more controversial: as AI-assisted programming becomes normal, language design will increasingly be judged not only by performance, ergonomics, or popularity, but by how legible it is to machines.

Keywords — code generation · LLM evaluation · programming languages · Elixir · Python documentation · benchmark analysis · software engineering

1 Introduction

Start with one striking number: 87.4.

That is Elixir's Pass@1 on AutoCodeBench, a benchmark of 3,920 automatically generated code tasks across 20 programming languages. Python — the default language of modern machine learning and one of the most represented languages in public code corpora — scores 43.9%. JavaScript scores 42.9%. Elixir, a language with tiny market share relative to both, nearly doubles them.

The first instinct is to call it noise. Perhaps Elixir received easier tasks. Perhaps the benchmark oversamples repositories with unusually polished documentation. Perhaps the effect is an artifact of prompt style or evaluation mechanics. The second instinct is to treat the result as a victory lap for functional programming. Both instincts are too shallow. The effect is large, but the explanation is subtler.

This paper begins from the AutoCodeBench leaderboard released by Tencent Hunyuan AI [1]. We reproduce the full 20-language evaluation with GPT-5.4 (medium reasoning), verify that Elixir remains a large positive outlier after difficulty controls, and then move beyond leaderboard description into causal investigation. The headline finding is clear: Elixir's edge is most consistent not with syntax minimalism or functional purity alone, but with a stack of design decisions that make program intent and task semantics unusually easy to recover from local context.

The most controversial claim of this paper is therefore *not* that Elixir is universally superior. It is that the next serious axis of language competition may be *model legibility*. Languages and ecosystems that make contracts, state transitions, and expected behavior explicit will enjoy a structural advantage in code generation, code review, and agentic software construction.

Contributions

This paper makes six concrete contributions.

1. **Reproduction with artifact control.** We reproduce the 20-language AutoCodeBench result and show that Elixir remains the largest positive outlier after difficulty-bucket controls and leave-one-language-out expectation analysis.
2. **Causal evidence from active ablations.** Across four completed active-ablation suites (A, D, E, F), the dominant within-Elixir signal is documentation structure. Removing examples alone does nothing; collapsing structured documentation cuts performance by roughly 40 pp.
3. **Cross-language portability analysis.** In a 144-row exact-task panel and 384-row fractional-factorial follow-up, explicit contract wording is the only portable intervention that survives Holm correction.
4. **A new conceptual formalization.** We introduce the *predictive burden* framework and a visibility-based interaction model that explains both Elixir's benchmark advantage and the diminishing marginal returns of further explicitness in already-high-visibility languages.
5. **A reframing of language design.** We argue that *LLM-friendliness* is becoming a real design axis, and we identify documentation architecture as a first-class part of that axis rather than an afterthought.
6. **Platform-level implications.** We connect the explicitness story to the BEAM virtual machine's actor model and discuss how the same design philosophy that makes Elixir legible for code generation may also align with agentic AI deployment requirements.

Outline. Section 2 reviews related work. Section 3 details the benchmark reproduction and artifact controls. Section 4 outlines our experimental design, and Section 5 maps the language design space. Section 6 presents causal evidence from active ablations; Section 7 reports exact-task multilingual studies. Sections 8–12 develop the interpretive framework, documentation-architecture comparison, practical implications, and platform-level discussion, and Section 13 concludes.

2 Related work

AutoCodeBench. AutoCodeBench introduces an automated pipeline for mining repository functions, generating natural-language tasks, constructing public and private tests, and evaluating code generation across 20 languages [1]. Its scale and multilingual coverage make it especially useful for comparative analysis, but also create a challenge: the tasks are not shared across languages, so raw leaderboard gaps cannot by themselves isolate causal language effects.

Functional-language evaluation. FPEval provides an independent control for the "maybe functional languages are just better for code LLMs" hypothesis [2]. In that study, more purely functional languages remain difficult for current models, which implies that Elixir's result is not explained by functional purity in the abstract.

Low-resource language expectations. The low-resource literature generally predicts the opposite of what we observe here: smaller languages should underperform because they are less represented in pretraining corpora [3]. The fact that Elixir dramatically outperforms Python therefore deserves special scrutiny.

Language confusion and popularity bias. Recent work documents both programming-language confusion in multilingual code generation [4] and a strong baseline preference for Python and familiar libraries [5]. In that context, Elixir's outperformance is doubly surprising: it wins despite being lower-resource and despite not being the obvious "default" language.

Community observations. The Elixir community quickly noticed the benchmark result, and practitioner commentary has emphasized documentation quality, local reasoning, and ecosystem consistency as candidate explanations [6, 7]. This paper turns those intuitions into a controlled empirical analysis.

BEAM runtime and agentic AI. Independent of the benchmark story, recent work has noted structural parallels between the BEAM actor model and emerging AI agent architectures. Guimarães argues that the actor model Erlang introduced in 1986 closely resembles the agent model being rediscovered for LLM orchestration [18]. NVLang provides additional evidence on BEAM language performance in code generation tasks [19]. We revisit these connections, cautiously, in Section 11.

3 Benchmark and headline result

3.1 The leaderboard

Table 1 shows the top and bottom of the reproduced 20-language leaderboard. Figure 1 presents the full ranking.

Unless otherwise noted, leaderboard numbers in this paper refer to the March 11, 2026 local fork reproduction with corrected host-native scoring paths. That snapshot retains 3,920 total rows but has slightly uneven per-language totals rather than a uniform 196 tasks. Elixir's reproduced slice contains 198 tasks; the active-ablation suites later in the paper reuse that same 198-task Elixir slice.

Table 1. Top 5 and bottom 5 languages by Pass@1 on the March 11, 2026 local AutoCodeBench reproduction.

Rank	Language	Pass@1	Hard%
1	Elixir	87.4	70.2
2	Kotlin	76.5	43.5
3	C#	72.4	46.2
4	Ruby	63.0	54.0
5	Julia	57.0	62.5
16	Python	43.9	68.4
17	Swift	43.5	60.5
18	Go	42.9	61.3
19	JavaScript	42.9	64.1
20	PHP	35.7	55.8

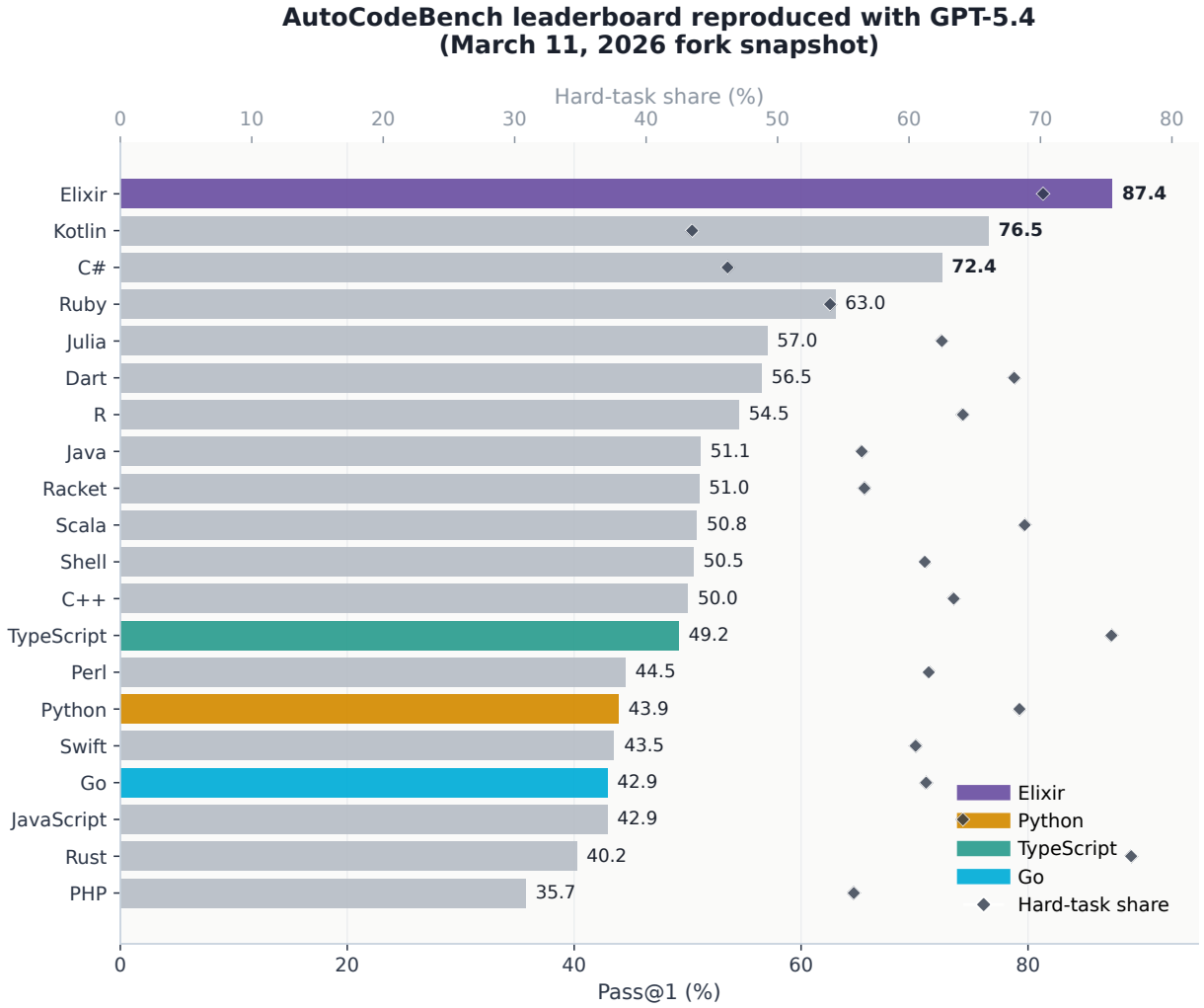

Figure 1. Full 20-language leaderboard. The bar chart shows Pass@1; the overlaid markers show each language's share of hard tasks. Elixir is both the top performer and one of the languages with the heaviest hard-task mix.

The first important point is simply that this is not a narrow win. Elixir leads second-place Kotlin by 10.9 pp and Python by 43.5 pp. The second important point is that this lead does not come from receiving unusually easy problems: Elixir's hard-task share is above the benchmark median.

The Go comparison. A particularly illuminating qualitative contrast is with Go. Both languages target concurrent, server-side workloads; both emphasize explicit error handling (Go via multiple returns, Elixir via tagged tuples); both favor relatively small core syntaxes and straightforward operational models. Yet Go scores 42.9% — identical to JavaScript and 44.5 pp behind Elixir. This paper does not directly measure Go's documentation architecture, so we treat the comparison as a hypothesis rather than a result: the Go–Elixir gap is consistent with the idea that explicitness at the *individual-function* level is necessary but not sufficient, and that architectural integration of contracts, documentation, and examples may matter more than any single feature.

3.2 Difficulty resilience

Table 2 and Figure 2 show the difficulty-bucket breakdown for five reference languages.

Table 2. Selected languages by difficulty bucket.

Language	Easy	Medium	Hard
Elixir	96.6	86.7	86.3
Kotlin	100.0	88.1	63.6
C#	97.8	81.1	63.1
Python	82.0	48.6	31.6
JavaScript	78.6	43.7	31.0

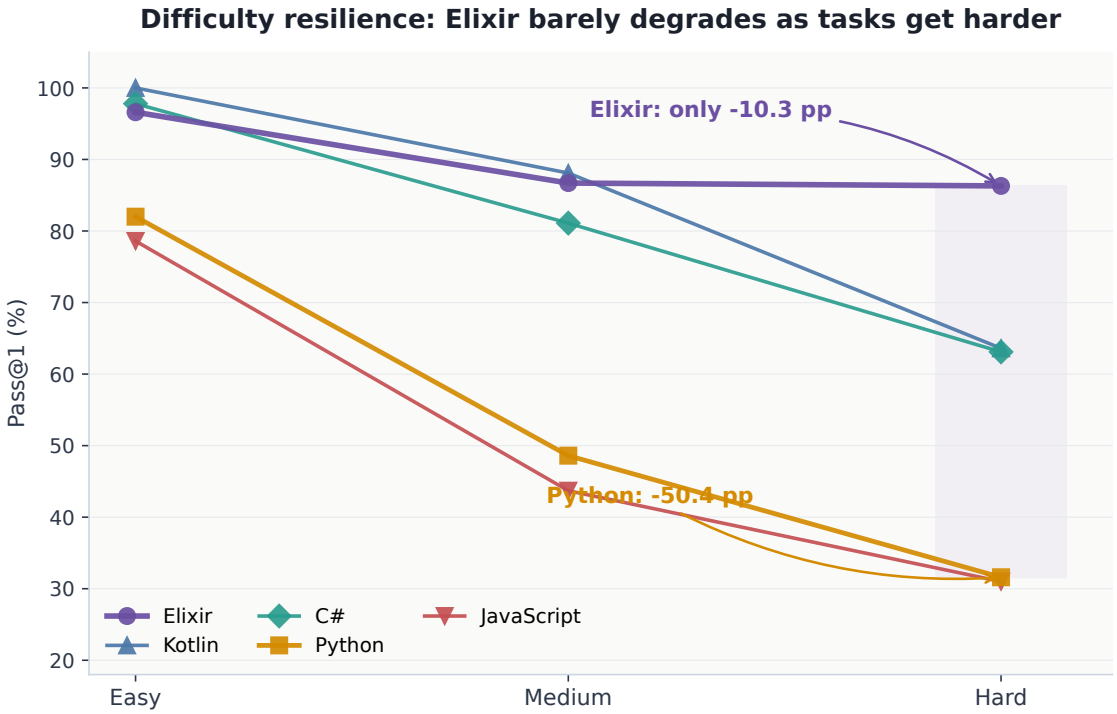

Figure 2. Elixir's edge is strongest where brittle generation should fail: hard tasks. Python drops by 50.4 pp from easy to hard; Elixir drops by only 10.3 pp.

This is a qualitatively important pattern. Elixir does not merely start high. It degrades unusually little as task difficulty increases. That is what one expects from a language-task combination that reduces ambiguity rather than from one that benefits from accidental benchmark quirks.

3.3 Artifact control

To test whether the leaderboard gap could be explained by task composition, we compute an expected success rate for each language by using the other 19 languages' pass rates within each difficulty bucket:

$$\hat{p}_{\text{expected}}(L) = \sum_{d \in \{E,M,H\}} \frac{n_d(L)}{N(L)} \cdot \bar{p}_d(\neg L). \tag{1}$$

For Elixir, the observed rate of 87.4% exceeds its difficulty-adjusted expectation of 44.7% by 42.7 pp. That is the largest positive residual in the dataset.

4 Design and methodology

4.1 Hypothesis suites

We organized the investigation into nine hypothesis suites, shown in Table 3.

Table 3. Hypothesis suites used to investigate the Elixir advantage.

Suite	Dimension	Method	Status
A	Documentation and task framing	Strip and degrade prompt-side documentation in Elixir tasks	Completed
B	Pattern matching scope	Proxy metric across 20 languages	Proxy only
C	Pipeline composition	Proxy metric across 20 languages	Proxy only
D	Control-flow style	Rewrite pattern-dispatch structures into alternative forms	Completed
E	Error-handling conventions	Replace tagged tuples with alternative return conventions	Completed
F	State and data flow	Vary pipeline, rebinding, and state-threading styles	Completed
G	Type-system signals	Proxy metric from specs and annotations	Proxy only
H	Difficulty-composition artifact	Leave-one-language-out expected-rate analysis	Completed
I	Repo-scale ecosystem effects	Full-project evaluation	Future work

Suites A, D, E, and F cover 792 suite-task slots and 2,970 condition-level evaluations in total. Suite H is the core artifact control. Suites B, C, and G provide descriptive context rather than causal claims.

The active-ablation suites operate on a fixed 198-task Elixir slice drawn from the same local fork snapshot used for the leaderboard reproduction. That matched-slice design is intentional: all within-suite deltas compare conditions on identical tasks, but the resulting `full_docs` baseline (84.3%) should not be read as a second estimate of the 87.4% full-benchmark score.

4.2 Proxy metrics

We use three summary metrics to compare the 20 languages.

Control-flow explicitness

$$S_{\text{cf}}(L) = \alpha \bar{P}(L) + \beta \bar{D}(L) - \gamma \bar{B}(L), \tag{2}$$

where $\bar{P}(L)$ is mean pattern-match density, $\bar{D}(L)$ is pipeline or dispatch density, and $\bar{B}(L)$ is branch density.

Mutability burden

$$M(L) = \bar{A}(L) + \bar{U}(L) + \bar{W}(L), \tag{3}$$

where assignments, updates, and writes all contribute to hidden state that the model must track.

First-failure taxonomy Each failed generation is classified by its first non-passed state: syntax or compilation error, runtime exception, wrong output, timeout, or other runtime failure. Wilson intervals and Fisher exact tests are used for rate comparisons.

For Suites A, D, E, and F, condition effects are estimated from matched task outcomes and tested with two-sided exact McNemar tests, with Holm correction across all non-baseline intervention contrasts. For the 384-row factorial, each main effect is summarized as a matched mean delta and tested with a two-sided exact sign test on win/loss counts after excluding ties, with Holm correction across the four main effects.

4.3 Claim tiers

To avoid rhetorical overreach, the paper separates claims by evidence tier.

Table 4. Claim-level summary.

#	Claim	Evidence tier	Key evidence
1	Elixir's 87.4% survives artifact controls	Strong	Difficulty controls and +42.7 pp residual
2	Documentation structure is the dominant within-Elixir driver	Strong (causal)	Suite A: roughly -38 to -41 pp
3	Pattern-matching dispatch style is not independently significant	Strong (null)	Suite D: all effects within ±3 pp
4	Error-handling style is not independently significant	Strong (null)	Suite E: all effects within ±3 pp
5	State-flow style is suggestive but unestablished	Directional	Suite F: +5.1 to +5.6 pp
6	Explicit contracts are the main portable cross-language intervention	Moderate (causal)	Factorial: $\Delta = +0.359$, Holm-adjusted $p = 0.030$
7	The Explicitness Hypothesis is a useful unifying interpretation	Framework	Integrates all completed evidence

5 Design-space analysis

Before turning to the active ablations, it is useful to ask whether Elixir already looks unusual in the 20-language design space. Figure 3 answers yes.

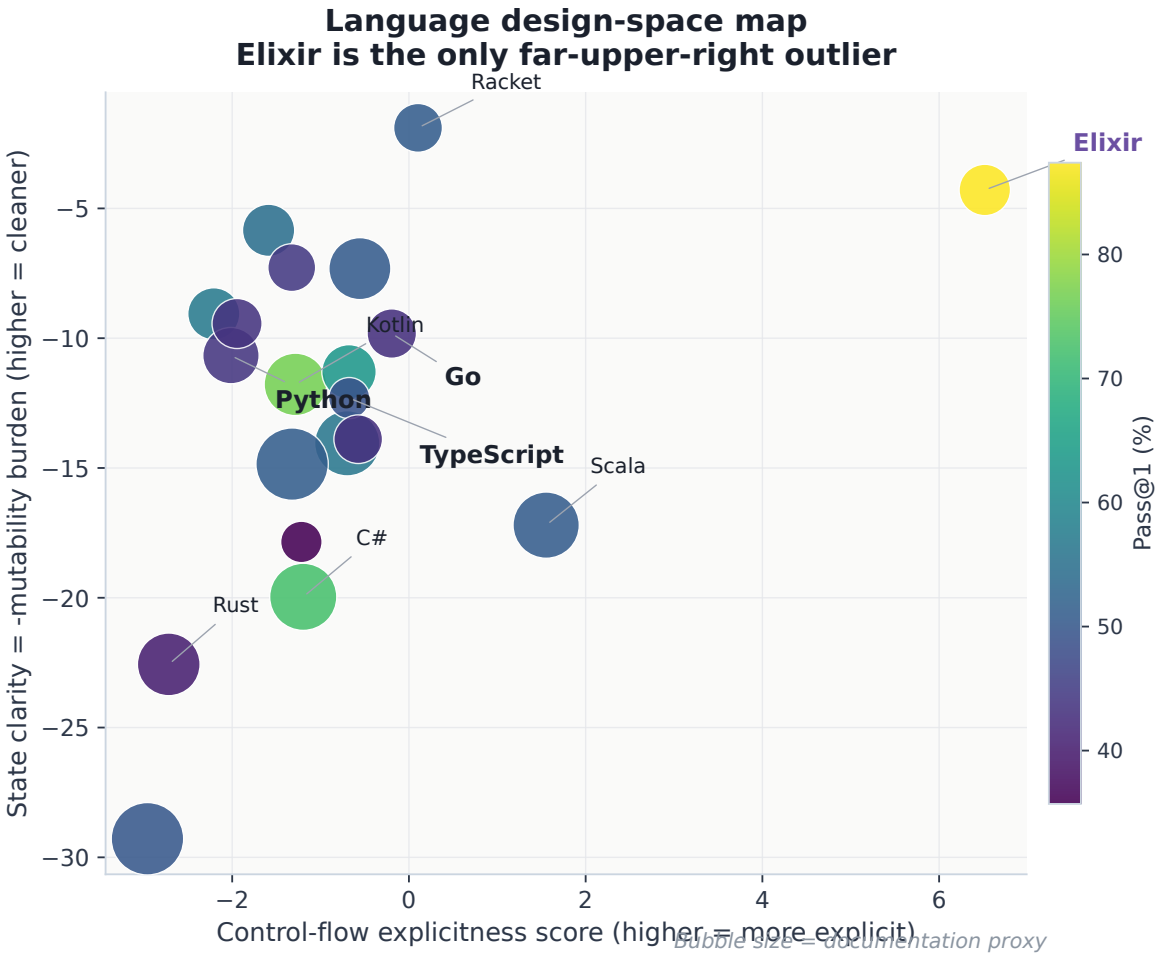

Figure 3. Design-space map built from the 20-language proxy table. The horizontal axis measures control-flow explicitness; the vertical axis is state clarity, defined as negative mutability burden so that higher is better. Bubble size reflects the documentation proxy. Elixir is the only far-upper-right outlier.

Three descriptive facts matter here.

First, Elixir is the only language with a strongly positive control-flow explicitness score (+6.517). Second, its mutability burden is near the bottom of the dataset. Third, its documentation proxy is not uniquely maximal — Java and C++ are higher on raw volume — which already hints that *structure* matters more than *quantity*.

The proxy analysis does not establish causality, but it does suggest a more precise target: Elixir combines explicit dispatch, low hidden state, and a documentation culture that appears unusually integrated with the code itself.

6 Causal evidence from active ablations

6.1 Suite A: documentation structure is the dominant driver

The strongest result in the paper is Suite A, shown in Table 5 and Figure 4. Because Suite A uses the fixed 198-task Elixir ablation slice described above, its `full_docs` baseline is the matched-slice baseline rather than the 87.4% full-benchmark figure.

Table 5. Suite A active ablation (198 tasks per condition).

Condition	Passed	Pass@1	Δ vs. full docs
full_docs	167	84.3%	—
reference_no_examples	167	84.3%	0.0
minimal_docs	91	46.0%	-38.4
signature_only	85	42.9%	-41.4

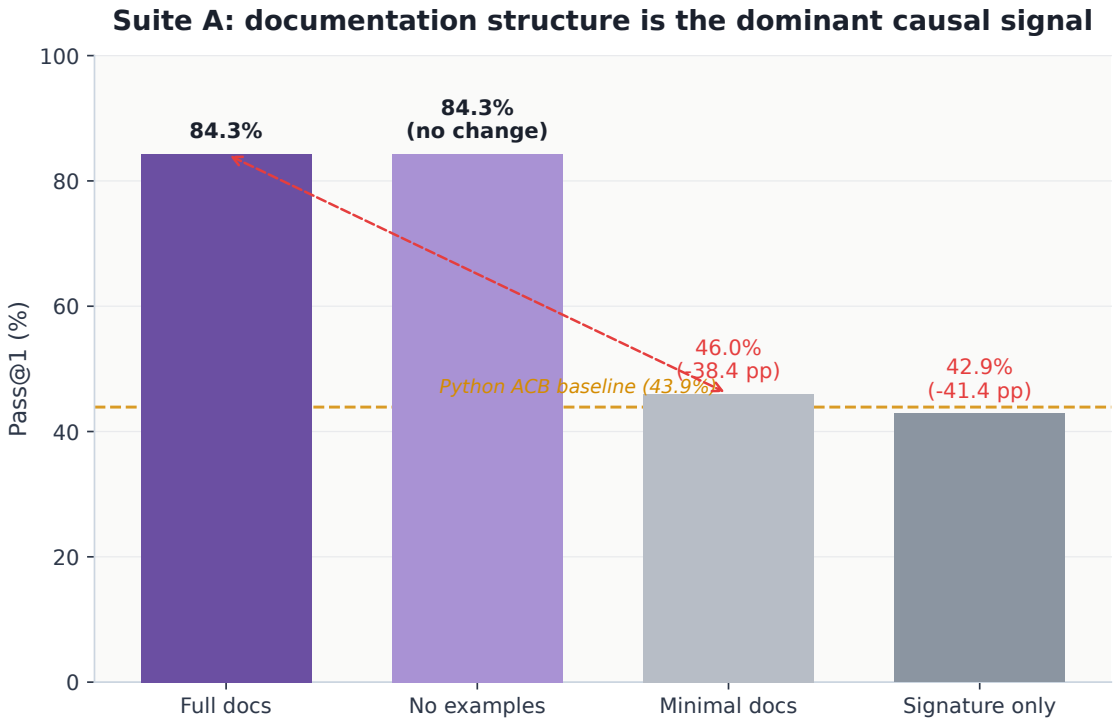

Figure 4. Within Elixir, examples alone do not matter, but collapsing documentation structure roughly halves Pass@1. Under signature-only framing, Elixir falls to the neighborhood of Python's overall ACB baseline.

Three points deserve emphasis.

1. **Examples by themselves do not move the benchmark.** Removing examples from the full Elixir framing does nothing.
2. **Structure is everything.** Removing structured documentation cuts performance by about 40 pp.
3. **Volume is not the key variable.** The gap between `minimal_docs` and `signature_only` is small. A little unstructured prose is not enough. What matters is a locally complete specification.

Both large drops remain Holm-significant under the matched McNemar analysis across all active-ablation contrasts.

That pattern is the central causal result of the paper. It reframes the Elixir question from "what syntax does the model like?" to "what information can the model recover without searching elsewhere?"

6.2 Other within-Elixir suites

Suites D, E, and F do not overturn the story; they sharpen it. Rewriting control flow away from pattern-dispatch structures does not produce large or significant effects. Rewriting error-handling conventions does not either. Alternative state-flow styles show small positive deltas, but these do not survive multiple-comparison correction. The full summary appears in Appendix B.

This negative evidence is valuable. It shows that the best explanation is not a single fetishized language feature. Elixir's advantage is not "the pipe operator won" or "tagged tuples won." The signal is concentrated higher in the semantic stack: documentation, contracts, and task framing.

6.3 Failure taxonomy and common-task sanity check

In the 198-task full-docs evaluation, there are only 25 failures. Of those, 7 are wrong-output failures, 2 are language-level exceptions, and 16 are other runtime or environment failures. The very small exception count is at least directionally consistent with a language that makes many control paths explicit rather than exception-driven.

A recurring-task fixed-effects sanity check also points in the same direction. On the small subset of recurring task titles where Elixir appears, the mean residual relative to the cluster mean is positive (+0.2143), although the interval is wide because the cluster count is small. This does not prove that topic composition is irrelevant, but it weakens the strongest purely observational objection.

7 Exact-task multilingual studies

7.1 144-row exact-task panel

The main leaderboard compares languages on different tasks. To partially neutralize that limitation, we constructed a 144-row exact-task panel: 16 hand-authored executable tasks, each implemented in Elixir, Python, and TypeScript under three framing conditions.

The aggregate condition results are:

Condition	Pass@1 (48 rows)
baseline_compact	35/48 = 72.9%
rich_contract	35/48 = 72.9%
rich_contract_examples	40/48 = 83.3%

Two points matter.

First, Elixir no longer dominates when all languages are given identical task specifications. That is important. It suggests that the AutoCodeBench result is partly an interaction between language properties and the benchmark's native framing style.

Second, examples become directionally helpful in this exact-task setting: 7 matched improvements versus 2 degradations. That is not contradictory to Suite A. In Suite A, examples were removed from a framing that was already rich and structured. In the exact-task panel, examples were *added* to compact specifications.

7.2 384-row fractional-factorial follow-up

The most informative same-task result comes from the 384-row regular $2^{(4-1)}$ fractional-factorial design, which varies four prompt-side factors: richer prose, examples, explicit contracts, and state-flow guidance. Main effects are estimated as matched deltas and tested with two-sided exact sign tests over win/loss counts, excluding ties, with Holm correction across the four factors.

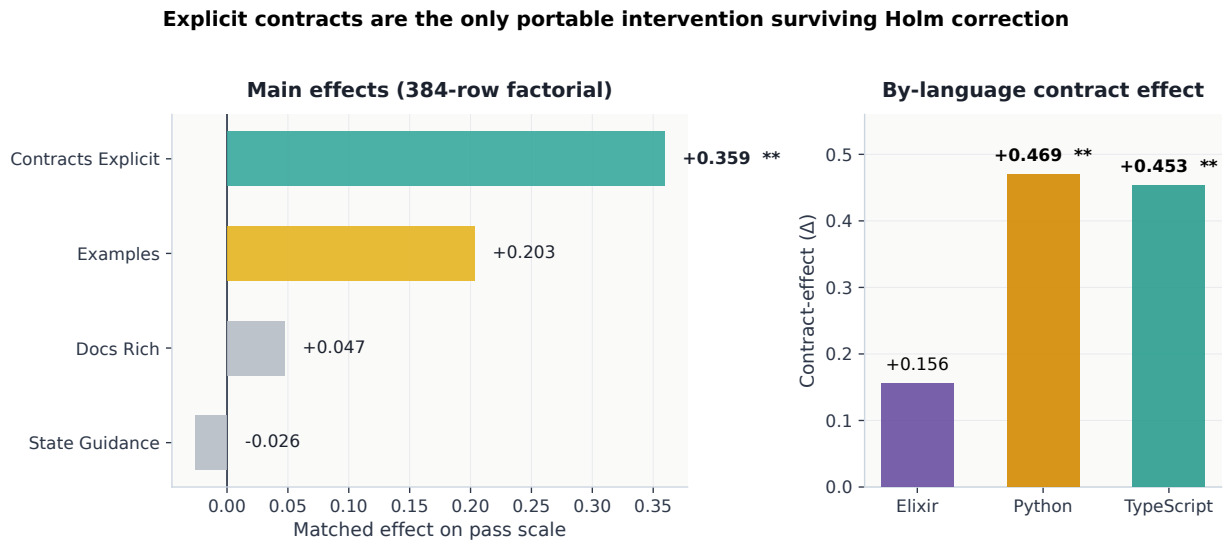

Figure 5. In the exact-task factorial study, explicit contract wording is the only portable intervention that survives Holm correction. Its gains are largest in Python and TypeScript, where comparable contract information is less native to the surrounding ecosystem.

Table 6. Main effects in the 384-row fractional-factorial study.

Factor	Matched Δ	Exact sign p	Holm-adjusted p
docs_rich	+0.047	0.549	1.000
examples	+0.203	0.039	0.116
contracts_explicit	**+0.359**	**0.007**	**0.030**
state_guidance	-0.026	0.727	1.000

By language, the contract effect is:

Language	Contract Δ	Holm-adjusted p
Elixir	+0.156	0.774
Python	**+0.469**	**0.001**
TypeScript	**+0.453**	**0.004**

This is precisely the kind of pattern a visibility-based explanation predicts. The languages that benefit most from explicit contract wording are the languages whose ecosystems do not already surface that information as aggressively and as locally as Elixir does.

8 Discussion: predictive burden, visibility, and benchmark alignment

The empirical picture is now crisp enough to formalize. The best interpretation of the data is not a one-feature story but a visibility story.

8.1 Local semantic visibility

Let

$$V(L,T) = \lambda_t E_{\text{task}} + \lambda_c E_{\text{contract}} + \lambda_f E_{\text{flow}} + \lambda_d E_{\text{dispatch}}, \tag{4}$$

where each E term captures the local visibility of a different semantic dimension for language L and task T:

- E_{task}: how clearly the task description states purpose and edge cases;
- E_{contract}: how clearly the input/output and error contracts are stated;
- E_{flow}: how visible state transitions and data transformations are;
- E_{dispatch}: how explicit control-flow selection is.

We then model success conceptually as

$$\Pr(\text{pass} \mid L,T) = \sigma(\alpha + \beta V(L,T) - \kappa D(T)), \tag{5}$$

where $D(T)$ is intrinsic task difficulty and $\sigma(\cdot)$ is the logistic function.

Equations (4)–(5) are *interpretive*, not fitted estimates. Their value is explanatory. They tell us what kind of mechanism can jointly explain the within-Elixir ablation, the exact-task multilingual follow-ups, and the weak role of several lower-level feature rewrites.

8.2 Benchmark alignment

The full AutoCodeBench result suggests more than a main effect. It suggests an interaction between language design and benchmark framing richness:

$$\Pr(\text{pass} \mid L,T) = \sigma(\alpha + \beta V_L + \gamma R_T + \delta V_L R_T - \kappa D_T), \tag{6}$$

where V_L is a language-level visibility prior and R_T is the richness of task framing.

This interaction model immediately explains the two most important empirical facts in the paper.

- On native AutoCodeBench tasks, Elixir wins big because both V_L and the benchmark's framing style are favorable.
- On exact-task panels, where framing is controlled and equalized across languages, Elixir no longer enjoys the same unique interaction bonus.

That is why the paper's real lesson is broader than "use Elixir." The benchmark appears to reward *aligned explicitness*.

8.3 A saturation result

The contract-effect asymmetry between Elixir and Python/TypeScript also falls naturally out of the visibility model.

Proposition 1 (Saturation of explicitness gains)**.** *Under Equation (5), the marginal gain from improving any visibility dimension E_k is*

$$\frac{\partial}{\partial E_k} \Pr(\text{pass} \mid L, T) = \beta \lambda_k P(1 - P), \tag{7}$$

where $P = \Pr(\text{pass} \mid L, T)$. Therefore, identical visibility improvements yield smaller absolute gains when baseline performance is already near ceiling.

Proof sketch. Differentiate the logistic form in Equation (5) with respect to E_k. The multiplier $P(1 - P)$ is maximal near $P = 0.5$ and shrinks as $P \to 0$ or $P \to 1$.

This matters empirically. Elixir already begins with strong local contract visibility, so the same explicit-contract intervention produces a smaller marginal benefit than it does in Python or TypeScript, where baseline contract visibility is weaker.

8.4 Mechanism summary

Table 7. Mechanism-level evidence summary.

Mechanism	Within-Elixir evidence	Cross-language evidence	Verdict
Documentation structure	+38 to +41 pp, Holm-significant in matched McNemar tests	Not tested portably	**Dominant driver**
Contract explicitness	Baseline already high	+0.359, Holm-significant exact-sign effect	**Portable intervention**
Control-flow style	≤ 3 pp, null	Not isolated	Not a primary driver
Error-handling style	≤ 3 pp, null	Not isolated	Not a primary driver
State-flow style	+5.1 to +5.6 pp, non-significant	Not isolated	Suggestive only

8.5 Relation to information density

The visibility account also connects naturally to the idea of uniform information density in human language processing [8]. Distributed, locally available information is easier to process than information that arrives in sparse, high-surprise bursts. In code, pattern-dispatch heads, explicit pipelines, nearby contracts, and executable examples all have the same general effect: they flatten semantic surprise. We do not claim to have measured token-level entropy here, but the theoretical connection is strong enough to motivate direct follow-up work.

9 Documentation architecture: why Elixir is structurally different

The empirical results repeatedly point upward, from syntax to architecture. What exactly is special about Elixir's documentation stack?

Table 8. Documentation architecture comparison.

Feature	Elixir	Python	TypeScript
In-source module docs	`@moduledoc` attribute	module docstring	no built-in first-class equivalent
In-source function docs	`@doc` attribute	function docstring	JSDoc or TSDoc comments
Executable examples	built-in doctest workflow	stdlib doctest exists but is unevenly used	no native equivalent
Types near docs	`@spec` adjacent to docs	type hints often separate from prose	types often separate from comments
Doc generation	ExDoc	Sphinx, pdoc, others	TypeDoc and related tools
Central hosting norm	HexDocs	fragmented, opt-in hosting	fragmented, opt-in hosting
Runtime retrieval	`Code.fetch_docs/1`, IEx help	`help()` via pydoc	editor and build-tool centric

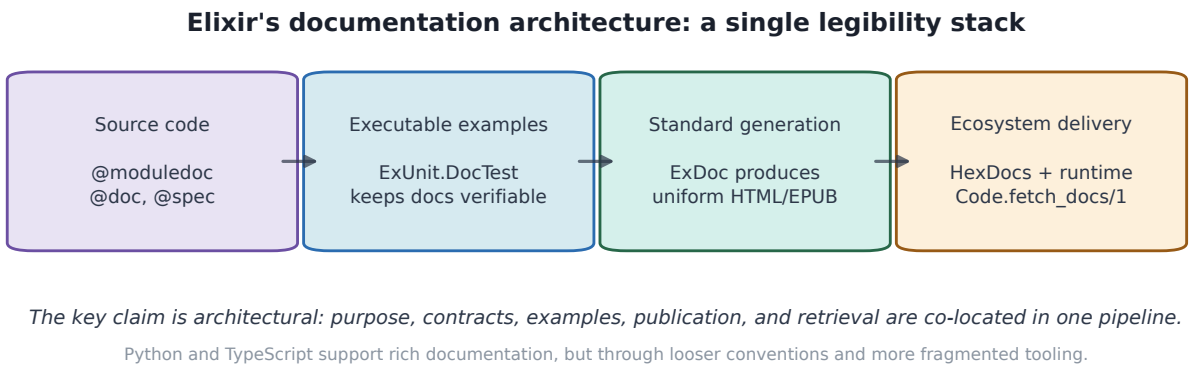

Figure 6. Elixir's advantage is architectural, not merely stylistic. The language, testing framework, doc generator, package ecosystem, and runtime help facilities form a single documentation pipeline.

The key difference is not that Elixir has more prose. Raw documentation volume does not explain the result. The difference is that purpose, examples, contracts, publication, and retrieval are all co-located in a stable pipeline. Source-level documentation attributes, executable doctests, ExDoc generation, HexDocs publication, and runtime retrieval via IEx or `Code.fetch_docs/1` are part of one system [9–13]. Python and TypeScript certainly support rich documentation, but they support it through looser conventions and more fragmented tooling [14–17].

That structural integration matters because code-generating models operate under context constraints. They benefit when the explanation of a function is physically and semantically nearby, not hidden in another file, another tool, or another social convention [20].

9.1 Why this is not just an Elixir vanity argument

A healthy version of the claim is portable. If explicit contracts produce the biggest cross-language gains in Python and TypeScript, then the paper is not telling teams to rewrite their stack in Elixir. It is telling them to reduce predictive burden wherever they already are:

- state input and output contracts explicitly;
- keep examples adjacent to the code they explain;
- make state transitions linear and named;
- prefer conventions that preserve local reasoning.

That is a stronger and more practical conclusion than language tribalism.

10 Implications

10.1 For language designers

If these results continue to replicate, language design will need a new question: *what does this language force into local view?* The immediate design lessons are simple.

1. **Make contracts first-class.** Return conventions, invalid-input behavior, and side effects should be visible from nearby syntax or nearby docs.
2. **Integrate documentation into the development loop.** Verified examples age better than ornamental examples.
3. **Prefer visible flow over hidden state.** Every extra mutation is another thing the model must silently remember.
4. **Treat doc architecture as language architecture.** If the ecosystem standardizes explanation, publication, and retrieval, the model sees a more regular world.

10.2 For software teams

The paper also yields a straightforward operational recipe.

- In Python or TypeScript, add explicit return and error contracts.
- Use a consistent documentation shape across the codebase.
- Keep examples executable whenever possible.
- Design functions so that a reviewer can understand them from the signature, nearby docs, and local body.

In plain English: write code so that neither a human nor a model needs to go on a scavenger hunt.

11 Beyond benchmarks: the BEAM platform and agentic implications

The evidence in this paper concerns code *generation*: given a prompt, how reliably does a model produce correct code? But Elixir's runtime — the BEAM virtual machine — has properties that make the story larger than benchmark scores alone.

11.1 The BEAM as an agent runtime

The Erlang/OTP platform, on which Elixir runs, was designed for telephone switches that must never stop. Its core abstractions — lightweight processes (each starting with approximately 2 KB of heap), preemptive

scheduling, per-process garbage collection, supervision trees, and transparent distribution — were originally motivated by fault tolerance [19]. Those same abstractions are at least suggestively aligned with several requirements of modern AI agent frameworks:

- **Lightweight isolation.** BEAM processes are cheaper than OS threads and cheaper than most language-runtime green threads. In practice, that supports very large numbers of concurrent isolated workers.
- **Fault isolation and recovery.** Each process has its own heap and crash boundary. A failing agent does not poison its neighbors; supervisors restart it according to a declared policy. This is the "let it crash" philosophy, and it maps directly onto the reliability requirements of agentic systems where individual tool calls or reasoning steps may fail.
- **Hot code reloading.** The BEAM supports upgrading running code without stopping the system. For long-running agent deployments, this means model updates, prompt changes, or tool registrations can be rolled out without service interruption.
- **Message passing as a first-class primitive.** BEAM processes communicate exclusively through asynchronous message passing. That closely resembles the agent-to-agent communication pattern in multi-agent frameworks, where agents exchange structured messages rather than sharing mutable state.

As Guimarães has observed, the actor model that Erlang introduced in 1986 bears a strong family resemblance to the agent model that AI is rediscovering today [18]. The supervision tree can be read as a policy engine; the process mailbox resembles an agent inbox; the GenServer callback protocol resembles a tool-use interface. The parallel is suggestive, not dispositive.

11.2 From legibility to liveness

This connection feeds back into the legibility story. A language that is easy for models to *generate* and also provides a runtime where generated agents can be *deployed, isolated, and recovered* may enjoy a compounding advantage. The documentation pipeline that makes Elixir legible to a code-generating model also makes the generated code self-documenting when it runs as an agent: `@moduledoc` and `@doc` attributes survive into production, and `Code.fetch_docs/1` allows agents to introspect their own or each other's interfaces at runtime [10].

11.3 What this paper does not claim

We do not claim that Elixir is the only viable agent runtime, nor that BEAM properties caused the benchmark scores observed here. The benchmark evaluation is purely about code generation from prompts; it does not test deployment, orchestration, or fault recovery. The argument is instead that the same design philosophy — make intent visible, keep state explicit, enforce isolation boundaries — manifests as legibility in code generation and as reliability in runtime execution. These are two faces of the same coin.

12 Limitations and future work

This paper is strong on one point and deliberately modest on several others.

1. **Single-model limitation.** All completed evaluations use GPT-5.4 with medium reasoning. A strong result should survive model-family variation.

2. **Benchmark specificity.** AutoCodeBench is large and useful, but it is still one benchmark with one task-generation pipeline.
3. **No shared task IDs in the main leaderboard.** The exact-task panels help, but they do not fully replace a truly shared-task multilingual benchmark.
4. **Small matched studies.** The 144-row panel and 384-row factorial are informative, but they remain small relative to the 3,920-task benchmark.
5. **Proxy metrics are approximations.** Our control-flow and mutability summaries are descriptive instruments, not direct measurements of language essence.

Future work should therefore do three things: scale the exact-task multilingual panels, replicate the active-ablation ordering on other frontier model families, and move beyond benchmark snippets into repo-scale comparisons among production frameworks (for example, Phoenix or Ecto versus comparable Django, Rails, or Express codebases).

Implications for new language design. Perhaps the most consequential implication of this study is what it suggests about languages that do not yet exist. The conventional assumption in LLM-assisted programming has been that *corpus size determines generation quality*: models perform best in languages with the most training data [3]. Elixir's result challenges that assumption directly. A language with tiny market share nearly doubles the score of the most represented language in public code corpora — not because models memorized more Elixir, but because Elixir's design makes intent recoverable from local context.

If the binding constraint is structural legibility rather than corpus volume, then the design space for new programming languages is wider than previously believed. A language engineered from first principles around the traits identified in this paper — explicit contracts, pattern-driven control flow, structured documentation as a first-class artifact, pipeline-oriented data transformation, and immutable-by-default semantics — could in principle match or exceed Elixir's LLM legibility without inheriting its ecosystem size limitations. The BEAM platform story (Section 11) adds a further dimension: pairing these generation-friendly design choices with an agent-native runtime may compound the advantage. The possibility has already motivated exploratory language-design work beyond the scope of this paper.[1] Whether purpose-built LLM-legible languages ultimately outperform evolved ecosystems is an empirical question this paper cannot answer, but the data presented here suggest that the question is now worth asking seriously.

13 Conclusion

We began with a striking result: Elixir scores 87.4% on AutoCodeBench, while Python scores 43.9%.

After artifact controls and causal follow-up, the cleanest interpretation is now visible. The result is not a simple morality play about functional programming. It is about *legibility*. Elixir performs unusually well because it often makes the function's purpose, contract, data flow, and failure modes readable from nearby context. When that structure is removed, performance collapses. When explicit contracts are added to weaker ecosystems, performance rises.

The durable claim of the paper is therefore larger than Elixir and sharper than hype: *LLM-oriented software engineering rewards explicitness*. Languages and ecosystems that reduce predictive burden will be easier for models to generate, repair, review, and extend.

[1] The author is separately exploring some of these design questions in DreamLang (https://dreamlang.dev). This note is included as a disclosure of relevant ongoing work, not as an evaluated artifact in the present study.

Moreover, the BEAM virtual machine's lightweight processes, fault-isolated supervision trees, and hot code reloading motivate a broader hypothesis, not tested directly here, about deployment as well as generation. A language that is both easy for models to generate *and* provides a platform where generated agents can be deployed, isolated, and recovered may enjoy a compounding advantage in agentic systems.

In that sense, the future language wars may not be decided only by speed, safety, or popularity. They may also be decided by who makes intent the easiest to recover — and which runtimes provide the best operational home for the agents that act on it.

* * *

Data and Code Availability

All task-generation scripts, prompt templates, execution harnesses, raw evaluation logs, and the manual audit pack referenced in this study are available at:

https://github.com/gunta/AutoCodeBenchmark

The repository includes the fractional-factorial design matrices, per-task pass/fail CSVs, and the figure-generation code needed to reproduce every table and plot in this paper.

A Full language proxy data

Table 9. Proxy metrics for all 20 languages, sorted by Pass@1.

Language	Pass@1	CF score	Mutability	Docs	Pattern	Assign.	Hard%
Elixir	87.4	6.517	4.289	17.694	3.545	4.970	70.2
Kotlin	76.5	-1.285	11.780	26.144	0.000	10.665	43.5
C#	72.4	-1.196	19.964	30.210	0.000	20.693	46.2
Ruby	63.0	-0.678	11.297	19.840	0.000	11.265	54.0
Julia	57.0	-2.208	9.060	18.166	0.000	9.540	62.5
Dart	56.5	-0.698	14.050	28.481	0.000	13.635	68.0
R	54.5	-1.588	5.844	18.255	0.000	5.389	64.1
Java	51.1	-1.322	14.851	35.082	0.000	12.410	56.4
Racket	51.0	0.103	1.897	15.944	0.000	1.051	56.6
Scala	50.8	1.553	17.202	29.427	0.000	17.653	68.8
Shell	50.5	-0.555	7.320	25.757	0.000	7.580	61.2
C++	50.0	-2.959	29.284	35.156	0.000	23.704	63.4
TypeScript	49.2	-0.676	12.308	11.356	0.000	10.271	75.4
Perl	44.5	-1.326	7.281	15.299	0.000	7.155	61.5
Python	43.9	-2.014	10.671	21.264	0.000	10.020	68.4
Swift	43.5	-1.945	9.434	16.783	0.000	10.220	60.5
Go	42.9	-0.194	9.821	16.338	0.000	9.885	61.3
JavaScript	42.9	-0.574	13.891	15.844	0.000	11.190	64.1
Rust	40.2	-2.718	22.566	26.343	0.000	19.070	76.9
PHP	35.7	-1.217	17.849	11.497	0.000	18.698	55.8

B Active ablation summary

Table 10. Active ablation conditions across Suites A, D, E, and F.

Suite	Condition	Passed	Total	Pass@1	Δ vs. baseline
A	full_docs	167	198	84.3%	—
A	reference_no_examples	167	198	84.3%	0.0
A	minimal_docs	91	198	46.0%	-38.4
A	signature_only	85	198	42.9%	-41.4
D	baseline	167	198	84.3%	—
D	case_with	161	198	81.3%	-3.0
D	cond_if	172	198	86.9%	+2.5
D	function_heads	164	198	82.8%	-1.5
E	baseline	166	198	83.8%	—
E	tagged_tuple_helpers	172	198	86.9%	+3.0
E	sentinel_helpers	166	198	83.8%	0.0
F	baseline	162	198	81.8%	—
F	immutable_pipeline	162	198	81.8%	0.0
F	explicit_state_threading	172	198	86.9%	+5.1
F	rebinding_stepwise	173	198	87.4%	+5.6

C Robustness checks for the matched studies

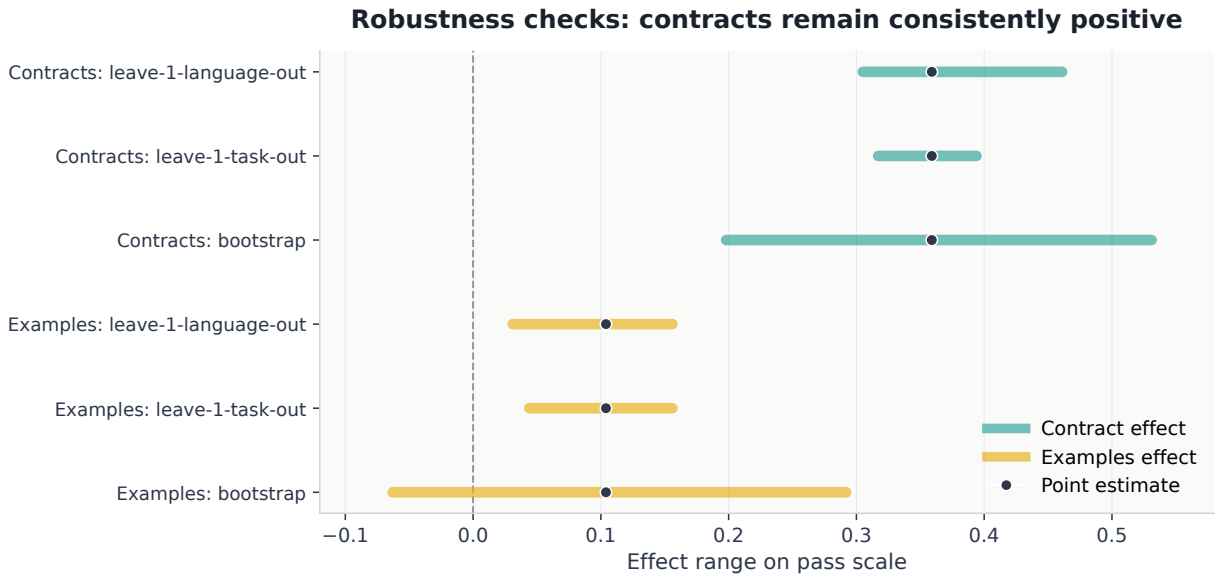

Figure 7. Quick robustness summary for the matched studies. The examples effect remains modest and low-power. The contract effect remains positive under both bootstrap and leave-one-out perturbations.

For the 16-task explicit panel, the `rich_contract_examples` minus `baseline_compact` paired delta is +0.104, with a task-cluster bootstrap 95% interval of [−0.063, +0.292]. Leave-one-task-out deltas range from +0.044 to +0.156, and leave-one-language-out deltas range from +0.031 to +0.156.

For the 384-row factorial, the main-effect robustness ranges are:

- `docs_rich`: bootstrap [−0.021, +0.125]; leave-one-task-out [+0.028, +0.061]
- `examples`: bootstrap [+0.089, +0.323]; leave-one-task-out [+0.172, +0.228]
- `contracts_explicit`: bootstrap [+0.198, +0.531]; leave-one-task-out [+0.317, +0.394]; leave-one-language-out [+0.305, +0.461]
- `state_guidance`: bootstrap [−0.099, +0.031]; leave-one-task-out [−0.033, −0.006]

D Task-level summary for the 16-task panel

Table 11. Outcome pattern in the 16-task explicit-task panel.

Task	Outcome pattern
Account Balance Rollup	Elixir fails under all three conditions; Python and TypeScript pass all three
Bracket Balance Report	Invariant across all three conditions
CSV Split With Quotes	Invariant across all three conditions
Dense Rankings	Rescued by examples for Elixir
Dependency Batches	Invariant across all three conditions
Inventory Reconcile	Invariant across all three conditions
Merge Touching Intervals	Invariant across all three conditions
Normalize Phone Book	Elixir fails under all three conditions; Python and TypeScript regress when examples are added
Queue Wait Times	Invariant across all three conditions
Rule Based Discount	Rescued by examples for Elixir and TypeScript
Session Durations	Rescued by examples for all three languages
Stable Group Runs	Rescued by examples for Elixir
Threshold Bursts	Invariant across all three conditions
Token Bucket Decisions	All three languages fail under all three conditions
Top K Frequencies	Invariant across all three conditions
Sliding Window Majority	Invariant across all three conditions

E Reviewer checklist

Table 12. Reviewer-concern map.

Reviewer concern	Mitigation in this version	Residual status
Main leaderboard is observational	Difficulty controls plus exact-task auxiliary studies	Narrowed, not eliminated
Matched-task panel may be too small	16 tasks, paired tests, task appendix, robustness checks	Still small
Matched conclusions may be one-task artifacts	Bootstrap and leave-one-out scans	Materially improved
Factorial may be fragile	Holm correction plus robustness ranges	Scoped, not universal
Elixir may simply have easier tasks	Leave-one-language-out expectation and hard-task controls	Materially improved
Failure analysis may be noisy	First-failure taxonomy and manual audit pack in repository	Partially improved
Paper may overclaim secondary mechanisms	Claim-tier table and explicit null results	Addressed

References

[1] Jason Chou, Ao Liu, Yuchi Deng, Zhiying Zeng, Tao Zhang, Haotian Zhu, Jianwei Cai, Yue Mao, Chenchen Zhang, Lingyun Tan, Ziyan Xu, Bohui Zhai, Hengyi Liu, Speed Zhu, Wiggin Zhou, and Fengzong Lian. AutoCodeBench: Large Language Models are Automatic Code Benchmark Generators. *arXiv preprint arXiv:2508.09101*, 2025.

[2] Nguyet-Anh H. Lang, Eric Lang, Thanh Le-Cong, Bach Le, and Quyet-Thang Huynh. Perish or Flourish? A Holistic Evaluation of Large Language Models for Code Generation in Functional Programming. *arXiv preprint arXiv:2601.02060*, 2026.

[3] Sathvik Joel, Jie JW Wu, and Fatemeh H. Fard. A Survey on LLM-based Code Generation for Low-Resource and Domain-Specific Programming Languages. *arXiv preprint arXiv:2410.03981*, 2024.

[4] Micheline Bénédicte Moumoula, Serge Lionel Nikiema, Abdoul Kader Kabore, Jacques Klein, and Tegawendé F. Bissyande. Programming Language Confusion: When Code LLMs Can't Keep their Languages Straight. *arXiv preprint arXiv:2503.13620*, 2025.

[5] Lukas Twist, Jie M. Zhang, Mark Harman, Don Syme, Joost Noppen, Helen Yannakoudakis, and Detlef Nauck. A Study of LLMs' Preferences for Libraries and Programming Languages. *arXiv preprint arXiv:2503.17181*, 2025.

[6] Dashbit. Why Elixir is the Best Language for AI. Dashbit blog, 2025.

[7] Armin Ronacher. A Language for Agents. lucumr.pocoo.org, February 2026. https://lucumr.pocoo.org/2026/2/9/a-language-for-agents/.

[8] T. Florian Jaeger. Redundancy and Reduction: Speakers Manage Syntactic Information Density. *Cognitive Psychology*, 61(1):23–62, 2010.

[9] Elixir Documentation Team. Writing Documentation. HexDocs. https://hexdocs.pm/elixir/writing-documentation.html.

[10] Elixir Documentation Team. Code.fetch_docs/1. HexDocs. https://hexdocs.pm/elixir/Code.html#fetch_docs/1.

[11] ExUnit Documentation Team. ExUnit.DocTest. HexDocs. https://hexdocs.pm/ex_unit/ExUnit.DocTest.html.

[12] ExDoc Documentation Team. ExDoc. HexDocs. https://hexdocs.pm/ex_doc/ExDoc.html.

[13] Hex.pm Team. HexDocs. Hex package manager documentation. https://hex.pm/docs.

[14] Python Documentation Team. pydoc — documentation generator and online help system. Python standard library documentation. https://docs.python.org/3/library/pydoc.html.

[15] Python Documentation Team. doctest — test interactive Python examples. Python standard library documentation. https://docs.python.org/3/library/doctest.html.

[16] TypeScript Team. JSDoc Reference. Official TypeScript documentation. https://www.typescriptlang.org/docs/handbook/jsdoc-supported-types.html.

[17] TypeDoc Team. Overview. TypeDoc documentation. https://typedoc.org/documents/Overview.html.

[18] George Guimarães. The Actor Model is the Agent Model. Dashbit blog, 2026.

[19] Nguyet-Anh H. Lang, Eric Lang, Bach Le, and Quyet-Thang Huynh. NVLang: A Benchmark for Evaluating LLM Code Generation in Niche Languages on the BEAM. *arXiv preprint arXiv:2512.05224*, 2025.

[20] Zach Daniel. Context-Aware Documentation and LLM Output Quality. Blog post, 2025.